

Day 6 - Errors, Exceptions, Modules, and Packages

Day 6 — Errors, Exceptions, Modules, and Packages

Objectives

- Understand what an exception actually is, and why Python uses them so heavily instead of just returning error codes
- Handle errors gracefully with `try/except/else/finally`, understanding what each part is actually for
- Create your own custom error types when the built-in ones don't fit
- Understand modules (files of Python code you can reuse) and packages (folders of related modules)

What is an exception, really?

You've already seen several errors crash a program by this point — `IndexError`, `KeyError`, `TypeError`, `ZeroDivisionError`. Each of these is an **exception**: an object Python creates the instant something goes wrong, describing exactly what happened. By default, when an exception is created and nothing "catches" it, Python stops your program immediately and prints that exception's details to the terminal (the traceback you learned to read on Day 0) — no further lines of code run.

The crucial idea for today: you can choose to **catch** an exception yourself, examine or respond to it, and keep your program running instead of letting it crash. This is an enormous amount of what separates "a script that works until something slightly unexpected happens" from "a real program that behaves sensibly even when things go wrong" — which is exactly what's expected of code written for a job.

`try` / `except` — catching an error instead of crashing`

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("You can't divide by zero!")
    result = None

print("Program keeps running normally after this.")
```

Read this as: "try to run the code in the `try` block; if a `ZeroDivisionError` happens anywhere inside it, jump immediately to the matching `except` block instead of crashing, run that instead, and then continue the program normally afterward." Without the `try/except`, that same division would have completely stopped the program right there.

You can catch different exception types differently, and Python checks each `except` in order, top to bottom, using the first one that matches:

```
try:
    value = int(user_input)          # could raise ValueError if user_input isn't a valid number
```

```

    result = 10 / value          # could raise ZeroDivisionError if value is 0
except ValueError:
    print("That wasn't a valid number.")
except ZeroDivisionError:
    print("Can't divide by zero.")

```

You can also catch several exception types with one shared response, by grouping them in parentheses:

```

except (ValueError, TypeError) as e:
    print(f"Something was wrong with the input: {e}")

```

as `e` captures the actual exception object into a variable named `e`, so you can inspect it — for instance, `print(f"{e}")` usually shows you the human-readable message describing exactly what went wrong.

Why you should (almost) never write a bare `except:`

It's tempting, especially as a beginner frustrated by an error, to write:

```

try:
    do_something_risky()
except:          # DON'T DO THIS
    pass         # `pass` means "do nothing" -- silently ignore literally ANY problem

```

This is dangerous because it catches *every possible thing that could go wrong**, including bugs you never anticipated and never intended to hide — a typo that causes a `NameError`, a completely unrelated crash somewhere deep inside a library you're using — and silently swallows all of it without any explanation, leaving you utterly confused later about why your program isn't behaving as expected, with zero clues as to why.

Always catch the specific exception type(s) you actually expect and know how to sensibly respond to.

`else` and `finally` — the two less-common, but useful, parts`

```

try:
    value = risky_operation()
except ValueError:
    print("that failed")
else:
    print("this runs ONLY if the try block succeeded with no exception at all")
finally:
    print("this ALWAYS runs, whether there was an error or not -- great for cleanup steps")

```

`else` is for code that should run only after a successful `try`, but that you deliberately do *not* want wrapped in the same `try` — this way, if that follow-up code itself has a bug, you won't accidentally and confusingly catch that separate bug with the same `except` meant for a different problem. `finally` always runs no matter what, which makes it useful for guaranteed cleanup — though as you learned yesterday, `with` is usually the better tool specifically for resources like files.

Raising your own exceptions

You're not limited to catching exceptions Python raises automatically — you can deliberately raise your own, any time your own code detects a situation it considers an error, using the `raise` keyword:

```

def withdraw(balance, amount):
    if amount > balance:
        raise ValueError(f"insufficient funds: have {balance}, need {amount}")
    return balance - amount

```

```
withdraw(100, 500) # raises: ValueError: insufficient funds: have 100, need 500
```

This immediately stops the function (and, unless something catches it, the whole program) and reports the message you gave, exactly like any error Python raises on its own.

Custom exception types — your own, specific categories of error

For errors specific to your own program's logic, it's good practice to define your own exception type rather than reusing a generic built-in one like `ValueError` for everything. You do this by writing a small class (you'll learn classes properly on Day 8, but this specific pattern is simple enough to use today):

```
class InsufficientFundsError(Exception):
    pass # `pass` here just means "this class needs nothing extra beyond what Exception already provides"

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(f"need {amount}, have {balance}")
    return balance - amount
```

Why bother creating your own type instead of just using `ValueError`? Because it lets whoever calls your code catch *precisely* your specific error, without accidentally also catching unrelated `ValueErrors` raised by something completely different elsewhere in the same `try` block:

```
try:
    withdraw(100, 500)
except InsufficientFundsError:
    print("Please add funds to your account.")
```

Modules — every `.py` file is one

You've been writing single files so far. A **module** is simply any `.py` file, viewed as something that can be *imported* — meaning another file can gain access to the functions, variables, and classes defined inside it.

```
import math # gives you access to everything inside the math module
print(math.sqrt(16)) # 4.0 -- accessed via "math.", since we imported the whole module

from math import sqrt # pulls just ONE specific name directly into your file
print(sqrt(16)) # 4.0 -- no "math." prefix needed now

import math as m # "aliasing" -- giving an imported module a shorter nickname
print(m.sqrt(16)) # 4.0
```

`math` here is one of Python's **standard library** modules — code that comes bundled with every Python installation, ready to use, covering everything from mathematics to dates to file paths, without needing to install anything extra.

Avoid `from module import *` (which imports absolutely everything a module offers at once) in real code — it makes it unclear, when reading code later, exactly which module any given name actually came from, and it can silently overwrite names you already have without any warning.

Packages — a folder full of related modules

Once a project grows beyond a handful of files, you organize related modules into a **package** — simply a folder containing multiple `.py` files (and typically a special, often-empty file named `__init__.py`, which is

what tells Python "treat this folder as an importable package"):

```
myproject/  
  mypackage/  
    __init__.py  
    utils.py  
    models.py  
  main.py
```

From inside `main.py`, you could write `from mypackage import utils` or, to reach even deeper, `from mypackage.utils import some_function`. You'll build a small package for real yourself on Day 13.

`if \_\_name\_\_ == "\_\_main\_\_":` — a pattern you'll see in nearly every real Python file

Every module Python runs has a hidden variable called `__name__`. When you run a file **directly** (`python my_file.py`), Python automatically sets `__name__` to the exact text `"__main__"` inside that file. But if that same file is instead **imported** by some other file, `__name__` is set to the module's own name instead (e.g., `"my_file"`) — never `"__main__"`.

This lets a single file provide reusable functions AND optionally act as a runnable script, without the "run as a script" part happening by accident just because someone else imported it for its functions:

```
def main():  
    print("Running as a script!")  
  
if __name__ == "__main__":  
    main()
```

If another file does `import my_file`, that import alone will **not** print `"Running as a script!"` — `main()` only actually runs if `my_file.py` itself is the file you executed directly from the terminal. You'll use this exact pattern in tomorrow's mini project, and in essentially every standalone script you write from here on.

Exercises

Open `exercises.py`, fill in the `# TODOs`, and run `python exercises.py` to check your work.